VAI - Vehicle Accountability Index

Technical Requirements Document (TRD)

Field	Detail
Document	Technical Requirements Document, Version 1.0
Date	30 October 2026
Status	Draft for engineering review (prototype stage)
Owner	VAI Engineering Team
Companion document	Product Requirements Document (PRD) v1.0
Classification	Confidential

Contents

No.	Section
1	Introduction and Scope
2	System Architecture
3	Technology Stack
4	Frontend Architecture
5	Backend Architecture
6	Database Design
7	API Documentation
8	Authentication Flow
9	VAI Scoring Engine Design
10	Reward Calculation Logic
11	Transaction Processing Logic
12	Security Architecture
13	Third-Party Integration Requirements
14	Deployment Architecture
15	Testing Strategy
16	Scalability and Performance
17	Error Handling and Logging
18	Dummy Dataset Architecture

1. Introduction and Scope

This document describes how VAI will be built. It translates the requirements in the PRD into system components, data structures, interfaces, formulas and operational practices. Where the PRD states a product rule, this TRD states the exact implementation. Requirement IDs such as FR-A2 refer to the PRD.

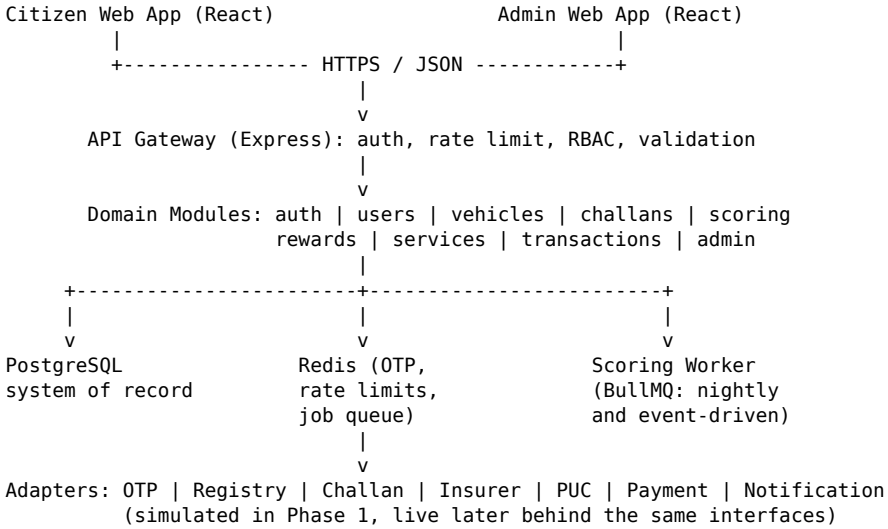
The prototype is a complete, runnable system on dummy data. Every external dependency (OTP delivery, vehicle registry, challan source, insurers, PUC centres and payment gateway) is behind an adapter interface with a simulated implementation. Replacing a simulated adapter with a live one must not require changes to the business logic, the frontend or the database schema.

1.1 Conventions

Convention	Meaning
Money	Stored as integer paise (1 rupee = 100 paise). No floating-point money arithmetic anywhere.
Rates	Discounts in basis points (1% = 100 bps). Platform fees in parts per million (ppm); 50 ppm = 0.005%.
Time	All timestamps stored in UTC as ISO 8601; displayed in Asia/Kolkata.
IDs	UUID v4 for internal keys; human-readable IDs for challans and transactions (e.g. TXN-20261030-000123).
Dates	Calendar days unless stated. Age calculations use integer days.

2. System Architecture

VAI follows a modular monolith architecture for the prototype and Phase 2. Modules have clear boundaries and communicate through service interfaces, so any module can be extracted into its own service later without rewriting the others. Background work such as nightly score recalculation runs in a separate worker process that shares the same codebase.



2.1 Component Responsibilities

Component	Responsibility
Citizen web app	Login, dashboard, vehicles, challans, score, rewards, services, transactions, profile
Admin web app	User and data views, configuration, audit logs, reports, dataset management
API gateway	Routing, authentication checks, rate limiting, input validation, request IDs, CORS
Domain modules	Business logic; the only layer that writes to the database
Scoring worker	Recalculates scores after events and nightly; writes ledger entries and snapshots
Adapters	Isolate simulated and live third-party systems behind common interfaces
PostgreSQL	Authoritative store for all business data and the audit trail
Redis	OTP sessions, rate limit counters, job queue, short-lived cache for score reads

3. Technology Stack

Layer	Choice	Reason
Frontend framework	React 18 with Vite and TypeScript	Component model suits dashboards; fast builds; type safety
Styling	Tailwind CSS with design tokens	Consistent design system; responsive utilities
Charts	Recharts for trends; custom SVG for the circular score meter	Lightweight; meter needs precise control
Client state and data	TanStack Query for server state; Zustand for session UI state	Caching and refetch handled for API data
Routing	React Router v6	Standard, well understood
Backend runtime	Node.js 20 LTS with TypeScript	Shared language with frontend; mature ecosystem
Web framework	Express 4 with helmet, cors, express-rate-limit	Requested stack; minimal and well documented
Validation	Zod schemas shared between frontend and backend	One source of truth for request and response shapes
ORM and migrations	Prisma with versioned migrations	Type-safe queries; repeatable schema changes
Database	PostgreSQL 16	ACID transactions for payments and ledgers; strong constraints; JSONB where needed
Cache and queue	Redis 7 with BullMQ	OTP storage, rate limits, reliable background jobs
Authentication	Signed JWT access tokens (15 min) and rotating refresh tokens (7 days)	Stateless verification with revocation via refresh table
Logging	pino with JSON output	Fast structured logs
Testing	Vitest, Supertest, Playwright, axe-core, k6	Unit, API, end-to-end, accessibility and load coverage
Containers and CI	Docker, docker-compose for local, GitHub Actions	Reproducible environments and automated checks

Note on MongoDB: the PRD does not mandate a database. PostgreSQL is chosen because the scoring ledger, transactions and fee calculations need relational integrity and multi-row transactions. A document store would require application-level enforcement of those guarantees.

4. Frontend Architecture

4.1 Routes

Route	Access	Purpose
/	Public	Landing page with product explanation and Get Started
/login	Public	Mobile number and PAN entry; requests OTP
/verify	Public (requires pending session)	OTP entry with countdown and resend control
/dashboard	Citizen	Score meter, category, counts, rewards summary
/score	Citizen	Score details, ledger, history chart, milestones
/vehicles	Citizen	Vehicle list with document status badges
/vehicles/:regNo	Citizen	Vehicle detail and vehicle-specific challans
/challans	Citizen	All challans with filters and sorting
/rewards	Citizen	Reward percentages and the user's current eligibility
/services/:type	Citizen	Renewal flow: vehicle, quote, breakdown, payment, receipt
/transactions	Citizen	Transaction history and receipt download
/profile	Citizen	Masked identity details, consent settings, logout
/admin/login	Admin	Admin sign-in with second factor
/admin/*	Admin roles	Admin modules; visibility controlled by role

4.2 Folder Structure

```
web/
  src/
    app/                routing, providers, layout shells (citizen, admin)
    features/
      auth/             login, OTP, session guard
      dashboard/        score meter, summary cards
      score/            ledger table, history chart, milestone card
      vehicles/         list, detail, document status badges
      challans/         table, filters, status chips
      rewards/          reward table, eligibility card
      services/         renewal wizard, price breakdown, payment simulator
      transactions/     history table, receipt view
      admin/            user, vehicle, challan, config, audit modules
    components/         shared UI: Button, Card, Badge, Table, Modal, Meter
    api/                typed API client generated from shared Zod schemas
    lib/                formatting (money, dates), masking helpers, a11y utilities
    styles/             Tailwind config, design tokens, theme variables
    tests/
```

4.3 State and Data Handling

Server data (vehicles, challans, scores, transactions) lives in TanStack Query with stale times of 60 seconds for scores and 5 minutes for reference data such as reward tables. Mutations invalidate the relevant query keys. Session state, which holds only the masked user summary and token expiry, lives in a Zustand store. Access tokens are kept in memory and refresh tokens are in an HttpOnly, Secure, SameSite=Strict cookie. The frontend never writes sensitive values to localStorage or sessionStorage.

4.4 Score Meter Specification

The circular meter maps score 500 to 900 onto a 270-degree arc. The arc colour changes by category: amber for Fair, teal for Good and green for Very Good. Each category band is marked on the arc at 650 and 750. The numeric value is shown in the centre with its category label beneath it. The meter has an accessible text equivalent, for example 'VAI Score 714, Good category, 36 points to Very Good', exposed through an aria-label and a visually hidden paragraph.

4.5 Responsive Breakpoints

Layouts are designed mobile first with breakpoints at 640px (sm), 768px (md), 1024px (lg) and 1280px (xl). Tables switch to stacked cards below 768px. Navigation becomes a bottom bar on mobile and a left sidebar from 1024px upward. Safe-area insets are respected on iOS devices.

5. Backend Architecture

5.1 Module Layout

```
api/
  src/
    server.ts                bootstrap, middleware order, graceful shutdown
    config/                  env schema (Zod), constants, feature flags
    middleware/              auth, rbac, validate, rateLimit, requestId, errorHandler
    modules/
      auth/                  OTP issue and verify, token issue and refresh
      users/                 profile, masked identity, consent records
      vehicles/              vehicle list and detail, document expiry status
      challans/              challan queries, dispute workflow
      scoring/               engine (pure functions), ledger writer, snapshot writer
      rewards/               category resolution, discount calculation
      services/              quote generation, renewal orchestration
      transactions/          state machine, fee calculation, receipts
      admin/                 user views, config versioning, audit log, reports
    adapters/
      otp/                   simulated SMS OTP; live adapter later
      registry/              dummy vehicle registry; VAHAN adapter later
      challan/               dummy challan source; eChallan adapter later
      insurer/ puc/ payment/ simulated providers with the same interface as live ones
    jobs/                    recalculation, nightly reconciliation, report generation
    db/                      Prisma schema, migrations, seed scripts
    lib/                     money, dates, crypto, masking, logger
  tests/
```

5.2 Request Lifecycle

Each request receives a request ID at the gateway. Middleware runs in this order: request ID, structured logging context, security headers, CORS check, body size limit (100 KB), rate limiting, token verification, role check, schema validation, controller, service, repository, response envelope. Services contain business rules and never read HTTP objects. Repositories contain query code and never contain business rules. This separation keeps the scoring engine testable as pure functions.

5.3 Configuration Management

Scoring weights, reward percentages and fee options are stored as versioned configuration rows, not in code. Each row has an effective-from timestamp. A calculation always uses the configuration version effective at the time of the event, so historical scores and transactions can be explained even after the rules change. Configuration changes are written to the audit log in the same database transaction as the change.

6. Database Design

The schema below is for PostgreSQL 16. Column names are in snake_case. Money columns are BIGINT holding paise. Enumerated values are implemented as PostgreSQL enum types or as CHECK constraints.

6.1 Entity Overview

```
users 1 ----< vehicles 1 ----< challans
  |                |
  |                +-----< vehicle_documents
  |
  +--1 vai_scores 1 ----< score_ledger
  +----< score_snapshots
  +----< licences
  +----< transactions >----1 service_providers
  +----< consent_records
  +----< disputes >----1 challans

reward_config, fee_config, scoring_config   (versioned, no user FK)
admins 1 ----< audit_logs >----1 (any entity via entity_type + entity_id)
otp_sessions                                  (short-lived, Redis-backed in production)
```

6.2 Table Definitions

users

Column	Type	Constraints and notes
user_id	UUID	Primary key
mobile_e164	VARCHAR(16)	Unique; stored in E.164 format
pan_encrypted	BYTEA	AES-256-GCM ciphertext including nonce and auth tag; never returned by API
pan_hash	CHAR(64)	Unique; HMAC-SHA256 of normalised PAN for lookup without decryption
pan_masked	VARCHAR(10)	Display form, for example XXXXXX3F9Q; the only PAN form the API returns
full_name	VARCHAR(120)	Not null
city	VARCHAR(80)	Not null
state	VARCHAR(80)	Not null
registration_date	DATE	Date the VAI account was created
status	ENUM	ACTIVE, SUSPENDED, CLOSED
is_dummy	BOOLEAN	True for all prototype records; enforced by a seed guard
created_at, updated_at	TIMESTAMPTZ	Default now(); updated_at set by trigger

vehicles

Column	Type	Constraints and notes
vehicle_id	UUID	Primary key
user_id	UUID	Foreign key to users; ON DELETE RESTRICT
registration_no	VARCHAR(16)	Unique; normalised uppercase without spaces
vehicle_type	ENUM	TWO_WHEELER, FOUR_WHEELER, COMMERCIAL
manufacturer	VARCHAR(60)	Not null
model	VARCHAR(60)	Not null
registration_date	DATE	Not null
insurance_expiry	DATE	Nullable; null means no policy on record
puc_expiry	DATE	Nullable
rc_validity_till	DATE	Not null

licences

Column	Type	Constraints and notes
licence_id	UUID	Primary key
user_id	UUID	Foreign key to users
licence_no_masked	VARCHAR(20)	Masked form only; full number is not stored in the prototype
licence_class	VARCHAR(20)	For example LMV, MCWG
expiry_date	DATE	Not null

challans

Column	Type	Constraints and notes
challan_id	VARCHAR(24)	Primary key; issuer-style ID such as CH-GJ-20260412-0001
user_id	UUID	Foreign key; denormalised for fast user queries
vehicle_id	UUID	Foreign key to vehicles
challan_date	DATE	Date of issue
challan_type	VARCHAR(40)	For example Traffic signal, Helmet, Speeding
violation_code	VARCHAR(20)	Used for repeat-violation grouping
violation_desc	VARCHAR(240)	Human-readable description
severity	ENUM	LOW, MEDIUM, HIGH
amount_paise	BIGINT	Positive; CHECK (amount_paise > 0)
status	ENUM	PAID, UNPAID, PENDING, DISPUTED
due_date	DATE	challan_date plus 60 days, unless the source supplies a due date
payment_date	DATE	Nullable; required when status is PAID
scoring_eligible	BOOLEAN	Derived: true for PAID or UNPAID, false for PENDING or DISPUTED

vai_scores

Column	Type	Constraints and notes
user_id	UUID	Primary key and foreign key to users
score	SMALLINT	CHECK (score BETWEEN 500 AND 900)
category	ENUM	FAIR, GOOD, VERY_GOOD; derived from score
score_before_cap	NUMERIC(7,2)	Raw total before the 500 to 900 clamp, kept for diagnostics
scoring_config_version	INT	Version used for this computation
computed_at	TIMESTAMPTZ	Last computation time

score_ledger

Column	Type	Constraints and notes
entry_id	BIGSERIAL	Primary key; append-only
user_id	UUID	Foreign key to users
event_type	ENUM	VIOLATION, REPEAT, PAYMENT, UNPAID_OVERDUE, STREAK, VEHICLE, LICENCE, DECAY, CAP_ADJ, INIT
component	VARCHAR(20)	V, RP, P, D, C or INIT
reason_code	VARCHAR(40)	Stable code for explanation text, for example VIO_MEDIUM_DECAYED
ref_id	VARCHAR(40)	Challan ID, vehicle ID or config version that caused the entry
delta_points	NUMERIC(7,2)	Signed change attributed to this entry
balance_before	SMALLINT	Score before entry
balance_after	SMALLINT	Score after entry
config_version	INT	Scoring configuration used
created_at	TIMESTAMPTZ	Insert time; rows are never updated or deleted

score_snapshots

One row per user per calendar month: user_id, period (first day of month), score, category, and a unique constraint on (user_id, period). Used for the previous-month comparison and the 12-month chart.

transactions

Column	Type	Constraints and notes
txn_id	VARCHAR(24)	Primary key; TXN-YYYYMMDD-nnnnnn
user_id	UUID	Foreign key to users
vehicle_id	UUID	Foreign key to vehicles
provider_id	UUID	Foreign key to service_providers
service_type	ENUM	INSURANCE, PUC, RC, LICENCE
vai_category_at_txn	ENUM	Category at the moment of quote
base_amount_paise	BIGINT	Original service amount
discount_bps	SMALLINT	Applied discount in basis points
discount_paise	BIGINT	Computed discount; rounded half-up
govt_funded_paise	BIGINT	Portion of discount funded by sponsor; equal to discount_paise in the prototype
payable_paise	BIGINT	base minus discount, the user's payable amount
platform_fee_ppm	INT	Fee option in parts per million: 50 or 100
platform_fee_paise	BIGINT	Computed fee on base_amount_paise; rounded half-up
status	ENUM	CREATED, VALIDATED, PAYMENT_PENDING, SUCCESS, FAILED, TIMED_OUT, REFUNDED
payment_ref	VARCHAR(40)	Simulated or gateway reference
created_at, updated_at	TIMESTAMPTZ	Timestamps; updated_at changes on every status transition

service_providers

provider_id, name, service_type, api_mode (SIMULATED or LIVE), settlement_account_ref (a reference token, never account numbers), fee_mode (PROVIDER_PAYS or PASS_THROUGH), is_active. In the prototype every provider is SIMULATED.

reward_config and fee_config

reward_config: id, vai_category, service_type, discount_bps, effective_from, created_by. Unique on (vai_category, service_type, effective_from). fee_config: id, option_code (A or B), ppm, mode (PROVIDER_PAYS or PASS_THROUGH), effective_from, is_active. Exactly one row may be active at a given time for each scope, enforced by a partial unique index.

scoring_config

version (INT, primary key), parameters (JSONB holding weights, caps, decay half-life and streak thresholds, validated against a schema), effective_from, created_by, change_reason. Rows are never modified; a new version is inserted instead.

admins, audit_logs, consent_records, disputes, otp_sessions

Table	Key columns and purpose
admins	admin_id, email (unique), password_hash (Argon2id), role (SUPER_ADMIN, OPS, ANALYST), mfa_secret_ref, is_active
audit_logs	log_id (BIGSERIAL), actor_type, actor_id, action, entity_type, entity_id, before_json, after_json, ip_hash, created_at. Append-only; the application role has no UPDATE or DELETE privilege on this table
consent_records	consent_id, user_id, purpose, granted (bool), version_text_hash, captured_at, withdrawn_at
disputes	dispute_id, challan_id, user_id, opened_at, resolved_at, outcome (UPHELD, REJECTED), resolver_admin_id, note
otp_sessions	session_id, mobile_hash, otp_hash (HMAC), expires_at, attempts, consumed_at. Stored in Redis in production with TTL

6.3 Relationships and Constraints

Deleting a user is a soft operation: status becomes CLOSED and personal fields are anonymised after the retention period, while ledger, transaction and audit rows remain for their legal retention period. Foreign keys to users and vehicles use RESTRICT so that orphaned financial records cannot exist. Transactions reference a vehicle and a provider at creation time; later changes to the vehicle do not alter historical transactions because amounts are stored, not recomputed.

6.4 Indexing Strategy

Index	Columns and type	Serves
users_mobile_uq	UNIQUE (mobile_e164)	Login lookup
users_pan_hash_uq	UNIQUE (pan_hash)	Login verification without decrypting PAN
vehicles_regno_uq	UNIQUE (registration_no)	Vehicle lookup
vehicles_user_idx	B-tree (user_id)	Vehicle list on dashboard
challans_user_date_idx	B-tree (user_id, challan_date DESC)	Challan list, newest first
challans_vehicle_idx	B-tree (vehicle_id, challan_date DESC)	Vehicle-specific history
challans_unpaid_partial	Partial B-tree (user_id, due_date) WHERE status = UNPAID	Overdue scan in nightly job
ledger_user_time_idx	B-tree (user_id, created_at DESC)	Score ledger page
snapshots_user_period_uq	UNIQUE (user_id, period)	Monthly comparison and chart
txn_user_time_idx	B-tree (user_id, created_at DESC)	Transaction history
txn_status_idx	B-tree (status, created_at)	Admin monitoring and reconciliation
audit_entity_idx	B-tree (entity_type, entity_id, created_at)	Audit search
score_distribution_idx	B-tree (category)	Admin distribution counts

6.5 Sample Records

```
users (one row, values illustrative)
{
  "user_id": "3f1c9a52-0b7e-4d8e-9a61-2c57e0d4b910",
  "mobile_e164": "+919000000001",
  "pan_masked": "XXXXXX1001X",
  "full_name": "Dummy User U001",
  "city": "Vadodara", "state": "Gujarat",
  "registration_date": "2024-03-14", "status": "ACTIVE", "is_dummy": true
}

challans (one row)
{
  "challan_id": "CH-GJ-20260412-0001",
  "user_id": "3f1c9a52-0b7e-4d8e-9a61-2c57e0d4b910",
  "vehicle_id": "8a0d2e7c-5f1b-4c3a-b8e9-1d6f7a2c9e44",
  "challan_date": "2026-04-12", "challan_type": "Traffic signal",
  "violation_code": "RLVD", "violation_desc": "Jumping red light",
  "severity": "MEDIUM", "amount_paise": 100000, "status": "PAID",
  "due_date": "2026-06-11", "payment_date": "2026-04-20", "scoring_eligible": true
}

transactions (one row, Very Good user, Rs. 10,000 insurance premium)
{
  "txn_id": "TXN-20261030-000123",
  "service_type": "INSURANCE", "vai_category_at_txn": "VERY_GOOD",
  "base_amount_paise": 1000000, "discount_bps": 200, "discount_paise": 20000,
  "govt_funded_paise": 20000, "payable_paise": 980000,
  "platform_fee_ppm": 50, "platform_fee_paise": 50, "status": "SUCCESS"
}
```

7. API Documentation

The API is versioned under /api/v1. Requests and responses are JSON. Citizen endpoints require a Bearer access token. Admin endpoints require an admin token with the matching role. Every response includes an X-Request-Id header. Errors use the envelope described in Section 17.

7.1 Endpoint Catalogue

Method	Path	Auth	Purpose
POST	/api/v1/auth/otp/request	None	Validate mobile and PAN; create OTP session; send OTP
POST	/api/v1/auth/otp/verify	None	Verify OTP; issue access and refresh tokens
POST	/api/v1/auth/refresh	Refresh cookie	Rotate refresh token; issue new access token
POST	/api/v1/auth/logout	Citizen	Revoke refresh token family
GET	/api/v1/users/me	Citizen	Masked profile and VAI category summary
GET	/api/v1/users/me/vehicles	Citizen	All vehicles linked to the user
GET	/api/v1/vehicles/{regNo}	Citizen (owner)	Vehicle detail with document status
GET	/api/v1/vehicles/{regNo}/challans	Citizen (owner)	Challans for one vehicle; filters and paging
GET	/api/v1/users/me/challans	Citizen	All challans; status, severity, date filters
GET	/api/v1/users/me/score	Citizen	Current score, category, previous month, milestone, top reasons
GET	/api/v1/users/me/score/history	Citizen	Monthly snapshots for the requested number of months
GET	/api/v1/users/me/score/ledger	Citizen	Paged score ledger entries
POST	/api/v1/rewards/eligibility	Citizen	Discount and breakdown for a service and amount
POST	/api/v1/services/quotes	Citizen	Create a priced quote; valid for 15 minutes
POST	/api/v1/transactions	Citizen	Confirm quote and run simulated payment
GET	/api/v1/transactions	Citizen	Transaction history
GET	/api/v1/transactions/{txnId}/receipt	Citizen (owner)	Receipt as JSON or PDF
GET	/api/v1/admin/users	Admin OPS+	Search users; masked PAN only
GET	/api/v1/admin/challans	Admin OPS+	Challan search across users
GET	/api/v1/admin/scores/distribution	Admin ANALYST+	Counts by category and score band
PUT	/api/v1/admin/config/scoring	Admin SUPER_ADMIN	Insert new scoring configuration version
PUT	/api/v1/admin/config/rewards	Admin SUPER_ADMIN	Insert new reward configuration rows
PUT	/api/v1/admin/config/fees	Admin SUPER_ADMIN	Activate fee option A or B with effective date
GET	/api/v1/admin/transactions	Admin OPS+	Transaction monitoring with filters
POST	/api/v1/admin/transactions/{txnId}/refund	Admin SUPER_ADMIN	Refund; reverses fee proportionally
GET	/api/v1/admin/revenue	Admin ANALYST+	Fee revenue by service, option and period
POST	/api/v1/admin/datasets/reset	Admin SUPER_ADMIN	Reset dummy dataset (blocked if is_dummy is false anywhere)
GET	/api/v1/admin/audit-logs	Admin ANALYST+	Search audit log; read-only

7.2 Sample Request and Response: OTP Verification

```
POST /api/v1/auth/otp/verify
{
  "sessionId": "b7e2c1d0-4f6a-4e2b-9c11-7a3d5e8f1a20",
  "otp": "482913"
}

200 OK
{
  "accessToken": "eyJhbGciOiJIUzI1NiIs...",
  "expiresIn": 900,
  "user": {
    "userId": "3f1c9a52-0b7e-4d8e-9a61-2c57e0d4b910",
    "fullName": "Dummy User U001",
    "panMasked": "XXXXXX1001X",
    "vehicleCount": 1
  }
}
```

The refresh token is set as an HttpOnly cookie in the same response and is not present in the JSON body.

7.3 Sample Request and Response: Score

```
GET /api/v1/users/me/score

200 OK
{
  "score": 714,
  "category": "GOOD",
  "previousMonthScore": 701,
  "changeSincePreviousMonth": 13,
  "computedAt": "2026-10-30T02:00:14Z",
  "nextMilestone": { "score": 750, "category": "VERY_GOOD", "pointsNeeded": 36 },
  "topReasons": [
    { "reasonCode": "STREAK_60D",       "label": "60 days without a violation", "delta": 10.0 },
    { "reasonCode": "VEH_FULL_COMPLY",  "label": "All documents valid on your vehicle", "delta": 15.0 },
    { "reasonCode": "PAY_ON_TIME_3",    "label": "3 challans paid by the due date", "delta": 9.0 }
  ],
  "configVersion": 4
}
```

7.4 Sample Request and Response: Reward Eligibility and Quote

```
POST /api/v1/rewards/eligibility
{ "serviceType": "INSURANCE", "vehicleId": "8a0d2e7c-5f1b-4c3a-b8e9-1d6f7a2c9e44",
  "baseAmountPaise": 1000000 }

200 OK
{
  "vaiCategory": "VERY_GOOD",
  "discountBps": 200,
  "breakdown": {
    "originalAmountPaise": 1000000,
    "discountPaise": 20000,
    "govtFundedPaise": 20000,
    "payablePaise": 980000
  },
  "platformFee": { "optionCode": "A", "ppm": 50, "feePaise": 50,
                   "chargedTo": "PROVIDER" },
  "quoteId": "Q-7H2K9P",
  "quoteExpiresAt": "2026-10-30T09:15:00Z"
}
```

7.5 Sample Request and Response: Confirm Transaction

```
POST /api/v1/transactions
{ "quoteId": "Q-7H2K9P", "simulatedOutcome": "SUCCESS" }

201 Created
{
  "txnId": "TXN-20261030-000123",
  "status": "SUCCESS",
  "payablePaise": 980000,
  "receiptUrl": "/api/v1/transactions/TXN-20261030-000123/receipt"
}
```

7.6 Pagination, Filtering and Sorting

List endpoints use cursor pagination with a limit between 1 and 100, defaulting to 20. Cursors are opaque base64 strings encoding the sort key and the last ID. Offset pagination is not used for ledgers, because rows are appended continuously. Filters use query parameters such as status=UNPAID&severity=HIGH&from=2026-01-01&to=2026-10-30. Sorting uses sort=challan_date:desc, with a whitelist of allowed fields.

8. Authentication Flow

8.1 Login and OTP Sequence

Step	Actor	Action
1	Frontend	Validates mobile format (E.164) and PAN format (5 letters, 4 digits, 1 letter); sends both to the API
2	API	Normalises PAN; computes pan_hash with HMAC-SHA256 using a server key; looks up the user
3	API	Returns the same response whether the user exists or not, to prevent account enumeration
4	API	If the user exists: generates a six-digit OTP using a CSPRNG, stores HMAC of the OTP with a 5-minute TTL, and calls the OTP adapter
5	OTP adapter	Simulated: writes the OTP to the server log in development only and shows it on the dummy-mode banner. Live: sends via DLT-registered SMS
6	Frontend	Shows OTP entry with countdown; allows resend after 30 seconds
7	API	Verifies OTP in constant time; increments attempts; on the third failure, invalidates the session
8	API	Issues access token (15 min) and refresh token (7 days, rotating, stored hashed)
9	Frontend	Stores access token in memory; redirects to /dashboard

8.2 Rate Limits

Scope	Limit	Window	Response on breach
OTP requests per mobile	3	10 minutes	429 with retryAfter
OTP requests per IP	10	1 hour	429
OTP verification attempts per session	3	Session lifetime (5 min)	Session invalidated
Authenticated API calls per user	300	1 minute	429
Admin login attempts per account	5	15 minutes	Account locked for 15 minutes

8.3 Token Handling

Refresh tokens are rotated on every use. The server stores a SHA-256 hash of each refresh token together with a family ID. If a refresh token that has already been rotated is presented, the entire family is revoked and the user must log in again, which limits the damage from a stolen token. Access tokens carry userId, role and an expiry; they contain no PAN or mobile number.

9. VAI Scoring Engine Design

The scoring engine is a set of pure functions. Given a user's confirmed records, the current date and the active configuration, it returns a score and a list of ledger entries. It does not read or write the database. The worker loads inputs, calls the engine, and writes results in a single transaction. Because the engine is pure, it is tested exhaustively with unit and property-based tests.

9.1 Inputs

Input	Source	Notes
Challans	challans table	Only rows with scoring_eligible = true
Vehicle document dates	vehicles table	Insurance, PUC, RC expiries per vehicle
Licence expiry	licences table	Latest licence per user
Last violation date	Derived from challans	Used for streak calculation
Current date	Server clock in Asia/Kolkata	Passed in explicitly for testability
Configuration	scoring_config (version)	All weights, caps and thresholds

9.2 Formula

The score is computed as follows, then clamped and rounded to the nearest integer for display, with the unrounded value stored in score_before_cap.

```
Score = clamp( 700 + V + RP + P + D + C , 500 , 900 )

V   Violation penalty (age-decayed, severity-weighted)
      V = max( -150 , - SUM over confirmed challans i of  W(sev_i) * 0.5 ^ (age_days_i / 365) )
      W(LOW) = 5     W(MEDIUM) = 15     W(HIGH) = 30
      Confirmed = status in (PAID, UNPAID). PENDING and DISPUTED are excluded.

RP  Repeat penalty (same violation_code within the last 365 days)
      RP = - min( 40 , 10 * max(0, n_same - 1) )
      n_same = count of confirmed challans sharing the violation_code in the window.

P   Payment behaviour
      P = min( 30 , 3 * N_on_time )  +  max( -100 , -20 * N_overdue_unpaid )
      N_on_time = PAID challans where payment_date <= due_date
      N_overdue_unpaid = UNPAID challans where today > due_date

D   Driving discipline (streak of days since the last confirmed violation)
      streak >= 30   adds +10
      streak >= 90   adds a further +15  (total +25)
      streak >= 180  adds a further +25  (total +50)
      D = applicable total, capped at +50

C   Vehicle and licence compliance
      Per vehicle v:  c_v = (+5 if insurance valid) + (+5 if PUC valid) + (+5 if RC valid)
                          + (-10 for each expired document among the three)
      C_vehicle = average over vehicles of c_v, clamped to [-30, +15]
      Licence expired at evaluation date:  additional -15
      C = C_vehicle + licence_term
```

9.3 Calibrated Parameters

Parameter	Value	Why it was chosen
Base score	700	PRD requirement
Score bounds	500 to 900	PRD requirement
Decay half-life	365 days	Violations lose half their weight after one year
Severity weights	5, 15, 30	A single High violation costs 30, under 5% of the range; one mistake is recoverable
Violation penalty cap	-150	Prevents total collapse from a large number of records
Repeat penalty step and cap	10 per repeat, max 40	Encourages breaking habitual patterns
On-time payment reward	3 each, max 30	Rewards timely payment without outweighing violations
Overdue penalty	20 each, max 100	Unpaid challans matter, but are capped
Streak milestones	30, 90, 180 days	Matches product milestones in the PRD
Vehicle compliance band	-30 to +15	Full compliance gives +15; fully expired vehicle gives -30
Monthly movement limit	50 points	Prevents sudden swings; applied as a final step

9.4 Decay and Recovery

Recovery happens through two mechanisms. The first is decay: each violation's influence falls by half every 365 days, so a violation from three years ago contributes about 12.5% of its original penalty. The second is the streak: every violation-free day counts toward the streak bonus. Together, they mean a user with one serious violation can recover the full range of a Good score within a year of clean driving, without the violation being erased from the ledger.

9.5 Worked Example

Inputs for user U001 variant on 30 October 2026: two LOW challans aged 200 and 300 days, one MEDIUM challan aged 60 days, all PAID on time; last violation 60 days ago; one vehicle with valid insurance, PUC and RC; valid licence. No repeated violation codes.

Component	Calculation	Value
V	-(5 x 0.5^(200/365) + 5 x 0.5^(300/365) + 15 x 0.5^(60/365))	-19.63
RP	No repeated violation codes	0.00
P	3 on-time payments x 3 = 9; no overdue unpaid	+9.00
D	Streak 60 days, which is at least 30 but under 90	+10.00
C	One vehicle, all three documents valid = +15; licence valid	+15.00
Total	700 + (-19.63) + 0 + 9 + 10 + 15 = 714.37, clamped and rounded	714

Category: GOOD (650 to 749). Points needed for VERY_GOOD: 36. Projection: a 90-day streak adds 15 more points through D alone, and continued payment and compliance supply the rest.

9.6 Processing Model

Trigger	Action	Latency target
Challan created or status changed	Event enqueued for that user; recalculation within minutes	Under 5 minutes
Vehicle document updated	Recalculation for that vehicle's owner	Under 5 minutes
Configuration version activated	Batch recalculation for all users in order of last change	Within 2 hours for 1 million users
Nightly job at 02:00 IST	Recalculate all users to apply decay and streak progression; write monthly snapshot on the 1st	Completes before 04:00 IST

9.7 Pseudocode

```
function computeScore(user, challans, vehicles, licence, today, cfg):
    ledger = []
    confirmed = [c for c in challans if c.status in (PAID, UNPAID)]

    V = 0
    for c in confirmed:
        age = days_between(c.challan_date, today)
        contribution = -cfg.W[c.severity] * 0.5 ** (age / cfg.half_life_days)
        V += contribution
        ledger.append(entry(VIOLATION, "V", "VIO_" + c.severity, c.challan_id, contribution))
    V = max(cfg.v_cap, V)

    RP = repeat_penalty(confirmed, today, cfg)
    P = payment_component(challans, today, cfg)
    D = streak_bonus(confirmed, today, cfg)
    C = vehicle_component(vehicles, licence, today, cfg)

    raw = cfg.base + V + RP + P + D + C
    score = clamp(round(raw), 500, 900)

    previous = last_score(user)
    if abs(score - previous) > cfg.monthly_move_limit:
        score = previous + sign(score - previous) * cfg.monthly_move_limit
        ledger.append(entry(CAP_ADJ, "-", "MOVE_LIMIT", config_ref(cfg), ...))

    persist_ledger_and_score_in_one_transaction(user, score, raw, ledger)
    return score
```

10. Reward Calculation Logic

10.1 Category Resolution

The category is derived from the most recent score, not from a cached label. Category boundaries are inclusive at the lower end: 500 to 649 is FAIR, 650 to 749 is GOOD, and 750 to 900 is VERY_GOOD.

10.2 Discount Rule

```
discount_bps   = lookup(reward_config, vai_category, service_type, effective_at = now)
discount_paise = round_half_up( base_amount_paise * discount_bps / 10000 )
govt_funded    = discount_paise                    # prototype: sponsor funds full discount
payable_paise  = base_amount_paise - discount_paise
```

Rounding is half-up to the nearest paisa. Integer arithmetic is used throughout: multiply first, then divide, and round once at the end, so that no intermediate rounding error accumulates.

10.3 Worked Example

Line	Formula	Amount
Original amount	Given	Rs. 10,000.00 (1,000,000 paise)
Discount	1,000,000 x 200 / 10,000	Rs. 200.00 (20,000 paise)
Government-funded benefit	Equal to discount in prototype	Rs. 200.00
User payable	1,000,000 - 20,000	Rs. 9,800.00 (980,000 paise)
Platform fee, option A	1,000,000 x 50 / 1,000,000	Rs. 0.50 (50 paise)
Platform fee, option B	1,000,000 x 100 / 1,000,000	Rs. 1.00 (100 paise)

10.4 Eligibility Edge Cases

Case	Handling
Score changes between quote and payment	The quote locks the category and discount for 15 minutes; payment uses the locked values
Category is FAIR	Discount is zero; breakdown still shows the line with zero value and the reason
Configuration changes mid-quote	Locked quote uses the configuration captured at quote time
Service not covered by any reward row	Discount is zero; the admin dashboard raises a configuration warning

11. Transaction Processing Logic

11.1 State Machine

```
CREATED --> VALIDATED --> PAYMENT_PENDING --> SUCCESS --> (REFUNDED)
   |            |                |
   v            v                +--> FAILED
(expired)    (expired)             +--> TIMED_OUT
                                         (only SUCCESS can be REFUNDED)
```

Transition	Trigger	Side effects
CREATED to VALIDATED	Quote is valid; vehicle is still owned by the user; provider is active	Reserve quote
VALIDATED to PAYMENT_PENDING	User confirms and submits payment	Create payment reference
PAYMENT_PENDING to SUCCESS	Simulated or gateway success	Update vehicle document status if renewal; issue receipt; record fee
PAYMENT_PENDING to FAILED	Simulated or gateway failure	Release quote; no fee recorded
PAYMENT_PENDING to TIMED_OUT	No response within 10 minutes	Mark for reconciliation; no fee until confirmed
SUCCESS to REFUNDED	Admin action	Reverse fee proportionally; restore vehicle status where applicable

11.2 Fee Calculation

The platform fee is computed on the original amount before discount. The fee rate comes from the active fee_config row at the time of quote. Fee mode determines who bears the fee: PROVIDER_PAYS deducts it from the provider's settlement, and PASS_THROUGH adds it to the user's payable amount as a separate line. In the default configuration the user payable is unchanged.

```
platform_fee_paise = round_half_up( base_amount_paise * ppm / 1000000 )
```

```
settlement_to_provider = base_amount_paise - discount_paise - platform_fee_paise   (PROVIDER_PAYS)
user_total = payable_paise + platform_fee_paise                                    (PASS_THROUGH)
```

11.3 Refunds

A refund reverses the transaction in full or in part. For a full refund, the platform fee is also reversed. For a partial refund, the fee reversal is proportional to the refunded share of the base amount, rounded half-up. Refund rows are separate records linked to the original transaction; the original row is never edited except for its status and updated_at. Refunds are restricted to SUPER_ADMIN and require a reason code recorded in the audit log.

11.4 Idempotency

Every POST to /transactions requires an Idempotency-Key header. The server stores the key with the response for 24 hours, and a repeated request with the same key returns the stored response without creating a second transaction. This protects against double charges caused by network retries.

12. Security Architecture

12.1 PAN Protection

Control	Implementation
Encryption at rest	AES-256-GCM with a random 96-bit nonce per record; key held in a KMS in production and in an environment secret for the prototype
Searchable reference	HMAC-SHA256 of normalised PAN with a separate key; used for lookup, never for display
Display	Masked form only (XXXXXX followed by the last four characters); API never returns full PAN
Logging	PAN fields are on a redaction list; logger replaces any match of the PAN pattern
Input handling	Full PAN entered at login is held in memory only, never persisted in plain form, and is excluded from request logs
Key rotation	Encryption keys versioned; re-encryption job supported; HMAC key rotation requires re-hashing

12.2 Access Control

Role	Permissions
CITIZEN	Own profile, own vehicles, own challans, own score, own transactions
ADMIN_ANALYST	Read-only aggregates, analytics, audit logs; no individual PAN or mobile visibility
ADMIN_OPS	Read individual user, vehicle and challan records (masked); transaction monitoring; dispute resolution
SUPER_ADMIN	All of the above plus configuration changes, refunds and dataset reset

Ownership checks are enforced in the service layer, not only in middleware. A request for /vehicles/{regNo} confirms that the vehicle's user_id equals the token's userId before any data is returned, and a mismatch returns 404 rather than 403 to avoid confirming that the vehicle exists.

12.3 Application Security Controls

Area	Control
Transport	TLS 1.2 or higher; HSTS with a one-year max-age; secure cookies only
Headers	helmet defaults; Content-Security-Policy restricting script sources; X-Frame-Options DENY
CORS	Allow-list of exact frontend origins; credentials allowed only for those origins
Input validation	Zod schemas on every endpoint; unknown fields rejected; body size limit 100 KB
SQL injection	Parameterised queries only through Prisma; raw SQL reviewed and parameterised
XSS	React escaping by default; no dangerouslySetInnerHTML; user text sanitised on input
CSRF	Refresh endpoint requires SameSite=Strict cookie and a custom header
Passwords	Admin passwords hashed with Argon2id; minimum 12 characters; breached-password check in production
Dependencies	Lockfiles committed; npm audit in CI; Dependabot alerts enabled
Secrets	Loaded from environment or secret manager; never committed; rotated on schedule
Audit	Append-only audit_logs; database role cannot update or delete audit rows

12.4 Privacy and Consent

Consent is captured in a dedicated step before the dashboard is shown. Each consent purpose (identity verification, vehicle data retrieval, challan data processing, scoring, and offers from partners) is recorded separately with the version of the notice shown. A user can withdraw consent for optional purposes from the profile page; withdrawal is logged and downstream processing stops within 24 hours. Personal data is minimised: services receive only the fields they need, and analytics use aggregated or pseudonymised data. Data retention and erasure follow the Digital Personal Data Protection Act, 2023 and the rules that apply to each data category once the legal review is complete.

13. Third-Party Integration Requirements

Every external system is accessed through an adapter with a defined interface. The prototype provides a simulated implementation for each adapter. No live integration is enabled until its approval, contract and security review are complete.

Integration	Purpose	Interface needed	Prerequisites before live use	Prototype
SMS OTP provider	Deliver one-time passwords	Send message; delivery callback	DLT registration of sender and template; provider contract	Simulated
VAHAN (Parivahan)	Vehicle registration, RC validity, owner details	Read vehicle by registration number	Government API access; authorization; consent; data-sharing agreement	Dummy registry
eChallan	Challan records, status and payment	List challans; receive status updates	API access; authorization; reconciliation agreement	Dummy challan source
Insurers	Quotes and renewal for insurance	Get quote; issue renewal; confirm policy	Insurer agreement; IRDAI product compliance; settlement terms	Simulated
PUC centres	Emission test booking and certificate	Check slot; book; receive certificate	Partner agreement; certificate verification method	Simulated
Payment gateway	Collect payment; settle to providers; refunds	Create order; capture; refund; webhooks	Licensed gateway; PCI DSS scope agreed; settlement account	Simulated

Integration	Purpose	Interface needed	Prerequisites before live use	Prototype
Consent manager	Record and manage consent artefacts	Create and revoke consent	Interoperable consent framework or in-house equivalent	In-house table
DigiLocker (optional)	Retrieve verified documents with consent	Pull document with user authorization	Partner onboarding; scope approval	Not in prototype

13.1 Adapter Contract Example

```
interface PaymentAdapter {
  createOrder(input: { txnId: string; amountPaise: number; payerRef: string }): Promise<OrderRef>;
  capture(orderRef: string): Promise<{ status: "SUCCESS" | "FAILED" | "PENDING"; reference: string }>;
  refund(reference: string, amountPaise: number): Promise<{ status: "REFUNDED" | "FAILED" }>;
}
// Implementations: SimulatedPaymentAdapter (prototype), GatewayPaymentAdapter (later)
```

The simulated adapter accepts a simulatedOutcome parameter so that QA can force success, failure or timeout. It never calls an external endpoint, and it refuses to run when the environment is set to production, to prevent accidental use of simulated payments on live data.

14. Deployment Architecture

Environment	Purpose	Hosting (suggested)	Data
Local	Developer machines	docker-compose with Postgres, Redis, API and web	Dummy seed only
Development	Integration builds	Single container host for API; static host for web	Dummy seed only
Staging	Demo, UAT, security testing	Managed Postgres; managed Redis; containers	Dummy seed only
Production (later)	Live service	Multi-AZ managed Postgres; autoscaling containers; CDN	Real data after approval

14.1 Pipeline

GitHub Actions runs on every pull request: type check, lint, unit tests, API tests against a test database, dependency audit and a secret scan. The main branch additionally builds container images, runs Playwright end-to-end tests, runs the accessibility checks, and deploys to staging. Production deployment requires manual approval and a migration dry run. Database migrations are forward-only, applied before the new code starts, and written so that the previous code version still works during rollout.

14.2 Operational Settings

Setting	Value
Runtime	Node.js 20 LTS, one process per container, graceful shutdown on SIGTERM within 20 seconds
Health checks	/healthz (liveness) and /readyz (database and Redis reachable)
Backups	Daily full backup; point-in-time recovery for 7 days in staging and 35 days in production
Environment variables	Validated at boot against a Zod schema; process refuses to start if invalid
Feature flags	Stored as configuration; used for rollout of live adapters

15. Testing Strategy

Level	Scope	Tools	Targets
Unit	Scoring functions, reward calculation, fee calculation, masking, money helpers	Vitest	Line coverage 90% or more on scoring, rewards and transactions
Property-based	Score always within 500 to 900; monotonic response to added violations; decay never increases a penalty	fast-check	Thousands of random histories per run
Golden tests	Worked examples from PRD and TRD reproduced exactly	Vitest snapshots	Must match to two decimal places
API integration	Every endpoint: success, validation failure, auth failure, ownership failure	Supertest with test Postgres	All endpoints covered
End-to-end	Login, dashboard, renewal flow, admin configuration change	Playwright	Critical journeys pass on desktop and mobile viewports
Accessibility	Each page against WCAG 2.1 AA rules	axe-core in Playwright	Zero serious or critical violations
Security	OWASP ASVS checks; auth bypass; IDOR attempts; rate limit behaviour	ZAP scan; manual review	No high-severity findings before pilot
Performance	Dashboard and score endpoints under concurrent load	k6	p95 under 300 ms at 200 concurrent users
Usability	Task completion by target personas, including older users	Moderated sessions	Renewal completed unaided by 80% of testers

15.1 Acceptance Criteria for Key Stories

Story	Acceptance test
US-01 login	Wrong OTP three times locks the session; correct OTP within five minutes logs in; expired OTP is rejected
US-02 score explanation	Every ledger entry has a reason code that maps to a plain-language label; sum of entries equals the change shown
US-05 discount before pay	Breakdown equals the worked example for Very Good, Good and Fair users
US-07 receipt	Receipt contains transaction ID, all breakdown lines and fee; regenerated receipt is identical
US-10 scoring config	New configuration applies only to events after its effective date; old scores remain explainable

16. Scalability and Performance

16.1 Targets

Phase	Registered users	Concurrent users	Read p95	Write p95
Prototype	Up to 10,000 dummy records	50	300 ms	500 ms
Pilot	Up to 500,000	5,000	300 ms	500 ms
Scale	Up to 50 million	100,000	300 ms	700 ms

16.2 Strategies

The API is stateless, so instances scale horizontally behind a load balancer. Score reads are served from Redis with a five-minute TTL and invalidated on recalculation. Challan and transaction lists use indexed cursor pagination, which keeps query cost stable as history grows. Scoring is performed asynchronously, so a large challan import never blocks a dashboard request. Reporting queries run against a read replica to protect transactional performance. Large tables such as the audit log and score ledger are partitioned by month once they exceed the pilot size.

16.3 Capacity Assumptions

Each active user generates on average 40 API calls per session. Each challan event triggers one scoring job taking under 50 milliseconds of CPU for a typical user with up to 200 records. A single worker process can therefore process more than 1,000 user recalculations per second, and the nightly batch for one million users completes well within its window with a small worker pool.

17. Error Handling and Logging

17.1 Error Envelope

```
{
  "error": {
    "code": "OTP_EXPIRED",
    "message": "This OTP has expired. Request a new one.",
    "requestId": "req_01J9ZK3W8Q4M7N2R5T6V8X0Y1A",
    "details": null
  }
}
```

17.2 Error Codes

HTTP	Code	Meaning
400	VALIDATION_FAILED	Request body or parameters did not match the schema; details list the fields
401	TOKEN_INVALID	Access token missing, malformed or expired
403	ROLE_FORBIDDEN	Authenticated but role not permitted for the action
404	NOT_FOUND	Resource not found or not owned by the caller
409	QUOTE_STALE	Quote expired or superseded; a new quote is required
409	TXN_STATE_INVALID	Requested transition not allowed from the current state
422	CONFIG_CONFLICT	Configuration overlaps an active effective period
429	RATE_LIMITED	Too many requests; retryAfter seconds returned
502	ADAPTER_UNAVAILABLE	A provider adapter failed; the request is safe to retry
500	INTERNAL_ERROR	Unexpected failure; logged with request ID, no internal detail returned

17.3 Logging Standards

Aspect	Standard
Format	One JSON object per line, with timestamp, level, service, requestId, userId (internal UUID only), route, status and duration
Levels	error for failed operations; warn for recoverable issues; info for business events; debug disabled in production
Prohibited content	PAN, full mobile number, OTP values, tokens, passwords, full challan description where it could identify a person
Correlation	Request ID propagated to adapters and to background jobs created by the request
Retention	Application logs 90 days; audit logs per legal requirement; security logs 1 year
Alerting	Alerts for error rate above 2%, transaction failure rate above 5%, scoring job lag above 15 minutes, and OTP abuse patterns

17.4 Failure Handling

Adapter calls use timeouts of 5 seconds and retries with exponential backoff and jitter, limited to three attempts, and only for idempotent operations. A circuit breaker opens after five consecutive failures and returns ADAPTER_UNAVAILABLE for 60 seconds. Background jobs are idempotent and keyed by user and configuration version, so a retried job produces the same ledger entries and does not duplicate them. Dead-letter queues hold jobs that fail five times and are visible in the admin dashboard.

18. Dummy Dataset Architecture

The prototype dataset is generated by a seed script that writes the ten profiles below into the same schema used in production. Every record carries is_dummy = true. The reset operation refuses to delete any row where is_dummy is false, so it cannot affect real data even if run by mistake. The seed is deterministic: the same seed value produces the same records and the same expected scores, which is essential for golden tests.

18.1 Identifiers and Format Rules

Field	Rule for dummy data
Mobile	+91 90000 00001 to +91 90000 00010 (reserved dummy range)
PAN	DUMMY1001X to DUMMY1010X; fails the real PAN structure check by design so it cannot be mistaken for a real PAN
Registration numbers	Format GJ01DM1001 style; prefix DM marks dummy; no real registration is used
Licence number	Stored only as masked form, for example GJ01-XXXX-4417
Names	Labelled as Dummy User U001 to U010 so that no real person is represented
Dates	Generated relative to the seed date of 30 October 2026

18.2 Profile Specification

User	Profile	Vehicles	Challan pattern	Expected category
U001	Zero challans	1 two-wheeler, all documents valid	None; long violation-free streak	VERY_GOOD (streak bonus and full compliance)
U002	Occasional minor	1 four-wheeler	3 LOW, all PAID on time, spread over 2 years	GOOD
U003	Frequent violations	2 vehicles	9 LOW and MEDIUM, repeated codes, 4 unpaid	FAIR
U004	Serious violations	1 four-wheeler	2 HIGH within 18 months, both PAID late	FAIR
U005	Unpaid challans	1 two-wheeler	4 UNPAID, 3 past due date	FAIR
U006	Excellent 12 months	1 four-wheeler	No violations in 180 days; 2 older LOW paid on time	VERY_GOOD
U007	Commercial fleet	3 commercial vehicles	Mixed; one vehicle with expired PUC	GOOD
U008	New registration	1 two-wheeler	None; account created 2 days ago	GOOD (starts at 700)
U009	Expired documents	2 vehicles, insurance and PUC expired	1 MEDIUM, PAID	FAIR
U010	Disputed and pending	1 four-wheeler	2 DISPUTED and 1 PENDING, excluded from score	GOOD

Expected categories are design targets for the golden test suite, not fixed scores. Exact scores depend on the seed date and the active configuration version.

18.3 Interconnection Rules

Every challan references one user and one vehicle, and the vehicle must belong to the same user. Every transaction references a user, a vehicle and a provider that exists. Every ledger entry references a configuration version that exists. The seed script validates these rules before committing and aborts if any rule fails. Dataset regeneration is available to SUPER_ADMIN from the admin dashboard, and each regeneration is recorded in the audit log.